

JSON - freedom or chaos

How to trust your data

A real-world journey from chaos to confidence
using pydantic and pytest



I had a Dream

"A proper system, a source of records, modelled, validated, easy to use, API friendly..."

- No more "what value should this field be?"
- One source of records with trustworthy data.
- Data generated instead of manually crafted or copied.

It is going to be beautiful.

Bart Dorlandt



- Freelance Network Automation Solution Architect
- 20 years in network engineering and automation
- Last year I presented "Repos are like children - Parenting 101"
- Lives by the phrase: *There must be a better way*



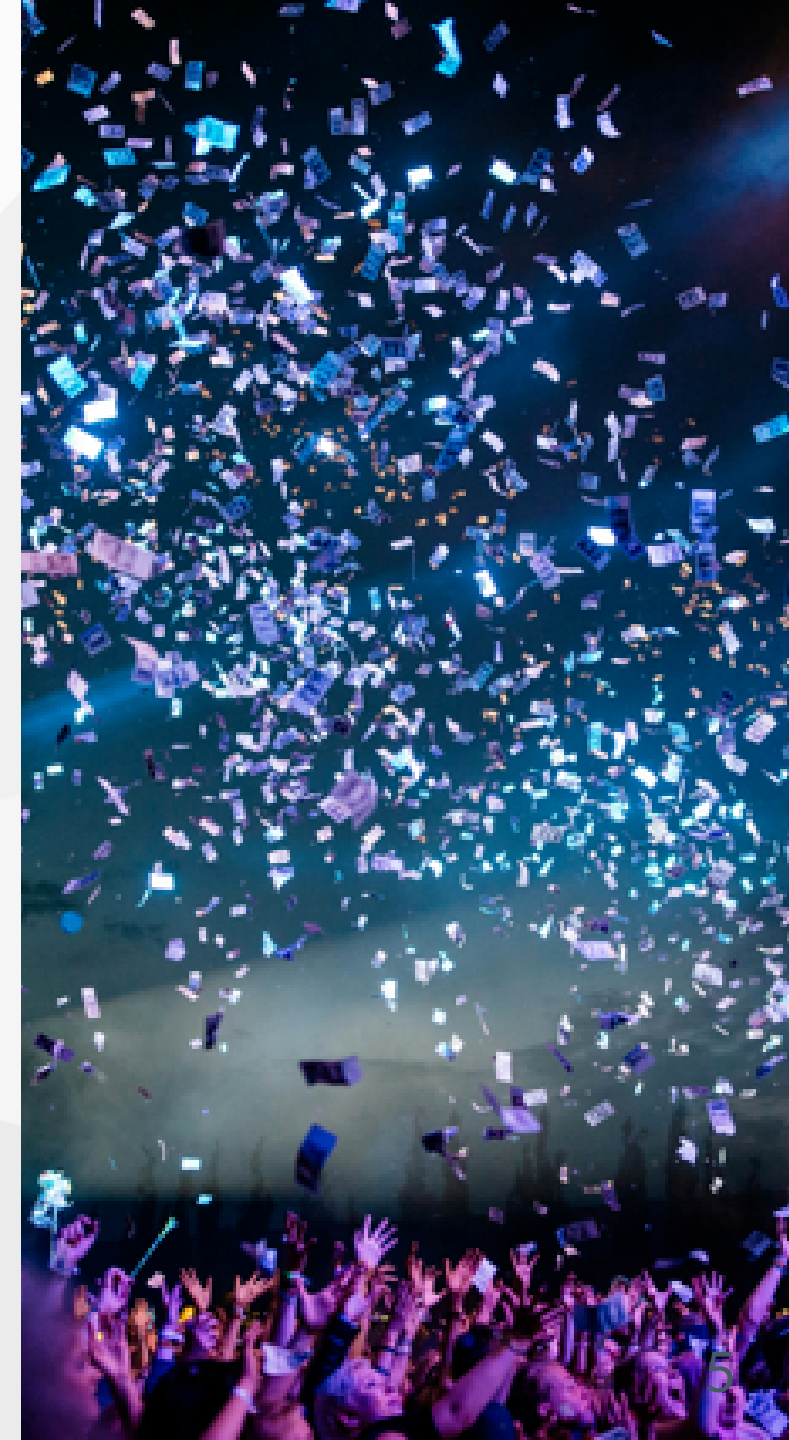
Where was I, Reality is a ...

"The solution was met with resistance."

- Fear of losing flexibility and freedom to change things quickly
- The jsons are in use, teams and tools depend on them
- Even having the same output generated wasn't enough...

What next?

- Some validation in place
 - Very limited data field validation
 - Mostly written on issue, "after the fact", when something was broken
- **No argument against improving validation!!**





Say no more

The situation

- Massive JSON files describing a multitude of sets of data
- Manually crafted (via Mako templates, or by hand)
 - Some dicts crafted on a single lines, others vertically aligned
- **12,000 lines** in "short" format — **72,000 lines** in long format
- Can't be modified and dumped programmatically
- Tools depend on it, some unknown unknowns might break on dumped JSON

The Freedom Trap

- Freedom data grows organically — it was convenient at the time
- No schema = no contract
- No contract = no trust
- Fields accumulate. Nobody removes them.
 - *Someone might be using it.*
 - Multi team challenges
 - Not everyone has the same skill set

Vision: A Better Future --> A Trusted Future

- Work within the constraints of the current situation
- Validate the changes through pipelines
- Validate the fields and the logic
- Ensure having it future proof, and maintainable in the long run

pydantic and pytest are our buddies (not only in this journey)

Meet "Freedom Data"

```
[
  "one_of_many_containers": {
    "type": "TYPE2",
    "active": "true",
    "networks": {
      "public": ["1.1.1.0/25"],
      "private": ["2.2.2.0/24"],
      "anycast": ["192.168.2.0/24"]
    },
    "network-equipment": {
      "rtr": {"type": "Model123", "hostname": "rtr-v1-pop01", "ip": "172.16.1.1", "active": "false"},
      "rtr2": {"type": "Model123", "hostname": "rtr2-v1-pop01", "ip": "172.16.1.2", "rack": "rack-1"},
      "sw": {"type": "Model456", "hostname": "sw-v1-pop01", "ip": "172.16.1.3", "role": "core"},
      "sw2": {"type": "Model456", "hostname": "sw2-v1-pop01", "ip": "172.16.1.4", "role": "core", "ports": ["te-0/0/1", "te-0/0/2"]},
    },
    "some_field_nobody_remembers": "value from 2014",
    "legacy_thing": true,
    "old_generation": "true",
    "connection_size": "100"
  }
]
```

"It works, don't touch it."

What is Pydantic?

It's a Python library for data validation and settings management using Python type annotations.

It allows you to define data models with type hints, and it will automatically validate and parse data according to those models.

Pydantic example

```
from pydantic import BaseModel

user = {
    "name": "Bart Dorlandt",
    "age": 25,
    "location": "The Netherlands",
}
```

```
class User(BaseModel):
    name: str
    age: int
    location: str

u = User(**user)
```

```
print(u)
>>> name='Bart Dorlandt' age=25 location='The Netherlands'
```

A little deeper

```
from ipaddress import IPv4Address
from pydantic import BaseModel, Field, PositiveInt
from typing import Literal

class Server(BaseModel):
    hostname: str
    ip: IPv4Address
    role: Literal["web", "db", "cache"]
    ports: PortLayout1 | PortLayout2
    type_id: PositiveInt = Field(alias="type-id")
    bs_flag: bool | None = None
```

Pydantic useful types

- PositiveInt
- `pydantic-extra-types[pycountry]`
 - CountryAlpha2
 - Latitude, Longitude
 - MacAddress

Start building the model

```
p = {  
    "full_name": "Example Pipe",  
    "provider_name": "ExampleProvider",  
    "type": "peering",  
    "provider_vlan": 100,  
    "lag": {  
        "trunk": "abc"  
    }  
}
```

```
class ConnectionLagTrunk(BaseModel):  
    trunk: str = Field(description="The trunk associated with the LAG")  
  
class Connection(BaseModel):  
    domestic: bool | None = None  
    full_name: str  
    provider_name: StrNoSpaces  
    type: Literal["peering", "transit"]  
    provider_vlan: int = Field(ge=1, le=4094)  
    lag: ConnectionLagTrunk
```

Down the rabbit hole

```
def port_name_validator(value: str) -> str:
    port_name_re = re.compile(r"^(xe|ge|et)-\d+/\d+/\d+(:\d+)?$")

    if not value or value == "":
        raise ValueError("Invalid port name")
    elif not port_name_re.match(value):
        raise ValueError(f"Invalid port name: {value}")

    return value
```

```
PortName = Annotated[str, AfterValidator(port_name_validator)]
```

```
class PortLayout1(BaseModel):
    srv_type1: list[PortName] | None = None
    srv_type2: list[PortName] | None = None

    @model_validator(mode="after")
    def validate_one_srv_type(self):
        """Validate that exactly one srv_type is defined."""
        srv_types = [self.srv_type1, self.srv_type2]
        defined_types = [sw for sw in srv_types if sw is not None]

        if len(defined_types) != 1:
            raise ValueError(f"Exactly one srv_type must be defined, but {len(defined_types)} were found")

        return self
```

Understanding the data

Informative scripts were used to understand the fields and optionality

- Split the giant JSON into smaller, more manageable pieces
- Focus on one main type at a time
- Validate each piece separately
- Build up the full model incrementally

Making things smaller creates more celebration moments :)

One blob of many done...

- Having one portion done doesn't guarantee success for the rest.
 - The freedom is hunting us
- This is a repetitive process, one fix at a time
- This is the moment where discrepancies are found
 - Start strict



Dealing with discrepancies

- str vs Literal - "cache " vs "cache" / `true` and `"true"`
- Consistencies in IP address/interface/network types
- Consistencies in IPv6 format
- Explicit is better than implicit
 - Use (smart) unions to define strict variations instead of 1 class with a bunch of optional fields
- Collect issues/mismatches to deal with the organization later

Parsing the data

```
type_mapper: dict[str, BaseModel] = {
    "type3": model_type3.TYPE3,
    "type2": model_type2.TYPE2,
}
...
for d, d_data in json_.items():
    try:
        m: BaseModel = type_mapper[d_type](**d_data)
    except ValidationError as e:
        logger.error(f"\nError processing {d!r}")
        model_helper.validation_error_printer(d, e)
        continue
...
```

First part done



- Accomplished the entire data matching the model
- Next step
 - generating the data from the modelled data
 - diff the original and generated data to find discrepancies
 - Did we capture all the data/fields?

Parse and Dump

```
type_mapper: dict[str, BaseModel] = {  
    "type3": model_type3.TYPE3,  
    "type2": model_type2.TYPE2,  
}  
  
for d, d_data in json_.items():  
    m: BaseModel = type_mapper[d_type](**d_data)  
    model_json = m.model_dump_json(exclude_unset=True, by_alias=True)  
    parsed_model = json.loads(model_json)  
    diff = DeepDiff(d_data, parsed_model, ignore_order=True)  
  
    if diff:  
        d_errors[d] = diff
```

Gotchas of parsing the data

- I had split the giant json into smaller pieces using the types
 - Different model types using the type mapper
 - Different data objects within
- Treat it as a giant black box will give you hell

Gotchas of generating the data from the model

- Use `model_dump` with `by_alias=True` to ensure the output matches the original field names (`Field(alias=...)` in the model)
- Same for `exclude_unset=True` to avoid dumping fields that were not filled in the original data

Second part done ✓

- Accomplished the data matching the model
- Accomplished the model matching the data
- Side Bonus: JSON Schema generation from Pydantic models
- Next step
 - Can we force everybody to care as much as I do?
 - Forced model hygiene

Side bonus

JSON Schema generation from Pydantic models

```
m = EntireModel.model_json_schema()  
schema = Path("schema.json")  
schema.write_text(json.dumps(m, indent=2))
```

And load it in VScode (settings.json)

Model Hygiene - validation our own model



Model Hygiene

"The model tells a story about the data. Make sure it's not fiction."

```
@pytest.mark.parametrize("model_class,m_type",
    [
        (model_type2.TYPE2, "TYPE2"),
        (model_type3.TYPE3, "TYPE3")
    ],
)
def test_main_model_obsolete_fields(json_, model_class, m_type):
    containers = [data for data in json_.values() if data["type"] == m_type]
    issues = _check_obsolete_fields_deep(model_class, containers)
    assert not issues, (
        f"Fields {issues} in {m_type} model tree are optional and never populated "
        "- probably obsolete! Remove and run the test again to confirm!"
    )
```

And the other direction

- Previous code example showed obsolete optionals
- The same is done for mandatory optionals
- Having this as part of the pipeline ensures:
 - Not only the data is updated
 - But also the model is kept tight and accurate over time
- Don't forget about the `Literal` list

Third part done



- Accomplished the data matching the model
- Accomplished the model matching the data
- Accomplished model hygiene enforcement
- Next step
 - Validation beyond single fields
 - Conditional dependencies between fields

Pytest to go beyond pydantic

- As seen, pydantic is great for validating individual fields and their types
- Pytest helps us to validate more complex rules
 - conditional dependencies between fields
 - uniqueness of values across the dataset

Uniqueness validation example

```
def test_duplicate_hostnames(json_: dict):  
    for d, d_data in json_.items():  
        hostnames = json_recursive_lookup_by_key("hostname", d_data)  
        counter = Counter(hostnames)  
        duplicates = [item for item, count in counter.items() if count > 1]  
        assert len(duplicates) == 0, \  
            f"Duplicate hostnames in {d}: {' '.join(duplicates)}"
```

Using fixtures for reusable subsets

```
@pytest.fixture(scope="session")
def flavor20sets(json_: dict[str, dict]) -> dict[str, dict]:
    """Fixture to provide a dict of flavor 2.0 sets.

    Returns:
        a subset of the instances dict containing only POP3s of flavor 2.0.
    """
    return {d: data for d, data in json_.items() if data["flavor"] == 2.0}
```

```
def test_ips_g20(flavor20sets: dict[str, dict]) -> None:
    for d, d_data in flavor20sets.items():
        ...
```

Using parametrize for multiple data points

```
json_paths = [  
    mapper.get_env_path("prd"),  
    mapper.get_env_path("stg"),  
]  
  
@pytest.fixture(params=json_paths, ids=lambda p: str(relative_to_basedir(p)), scope="session")  
def json_fixture(request: pytest.FixtureRequest) -> dict[str, dict]:  
    """Fixture to ensure pydantic models are loaded before tests run.  
  
    Returns:  
        A dictionary mapping of the json for each environment.  
    """  
    return instances_mapper.load_instances_pop_types(request.param)
```

Pytest focus areas

- Fixtures are key for reusable subsets of data
 - Achieving small and simpler test functions
- Parametrize is just awesome and key for testing multiple data sets

Fourth part done



- Accomplished the data matching the model
- Accomplished the model matching the data
- Accomplished model hygiene enforcement
- Accomplished validation beyond single fields and complex dependencies

Don't forget to educate teams

Challenges

- Let's have a look at the challenges that come with this



The 10-Year Data Problem

"Let's clean up this field that hasn't been used since 2014."

Sounds simple. It's not.

- Who else is using this field?
- Is there a team in another business unit depending on it?
- Is there a script nobody maintains that breaks silently if this is gone?

The scary part of freedom data: you can't be certain.

The Cross-Team Challenge

Not everyone works the same way:

Team A	Team B
Loves git	"What's a branch?"
Python fluent	Uses Excel
CI/CD native	Manual deploy
Rebase expert	Rebase FML

Lessons Learned

- Pydantic handles the messy real-world types very well
- Having the model hygiene is more important than initially expected
- The 3-way validation triangle keeps the model honest
- Changing fields/data is harder than building the pydantic model!
- Proving that a field could never be used, doesn't mean others believe it

Call to Action

Look at your own "freedom data." Ask yourself:

1. **Can you trust it?** Would a bug in it cause an incident?
2. **Is there a contract?** Or is it implicit knowledge in people's heads?
3. **Who else uses it?** Do you actually know?
4. **What's the smallest step?** One Pydantic model. One test. One field.

Q&A

Thank you.

- Bart Dorlandt
- <https://linkedin.com/in/bartdorlandt/>
- <https://dreamnetworking.nl/>
- <https://github.com/bartdorlandt/>

